



TEXAS A&M UNIVERSITY
Engineering

A Neural Network Approach for Calibrating Memristor Crossbar Arrays

Mohammad Rezaeifar, He Chaoyi, Kamran Entesari, Linda Katehi



TEXAS A&M UNIVERSITY
Department of Electrical
& Computer Engineering

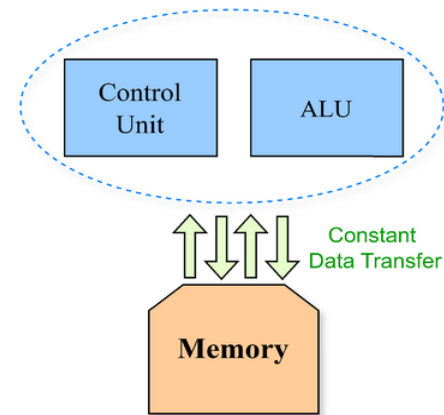
iEMSL
INTELLIGENT ELECTROMAGNETIC
SENSOR LABORATORIES

Title

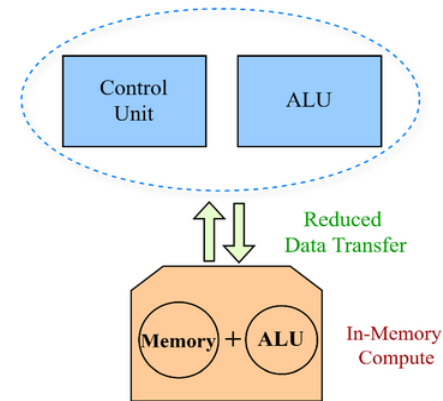
- **A Neural Network Approach for Calibrating Memristor Crossbars**
 - Mohammad Rezaeifar, He Chaoyi, Kamran Entesari, Linda Katehi
 - Texas A&M University
-

Motivation

- Von Neumann bottleneck → costly data movement
- In-Memory Computing (IMC) with memristors = promising solution
- BUT: crossbars suffer IR-drop, parasitic effects, non-linearities
- Accuracy degradation in analog MVM



(a) Depiction of the von Neumann architecture showing the performance-limiting data transfer bottleneck between memory and processing unit.

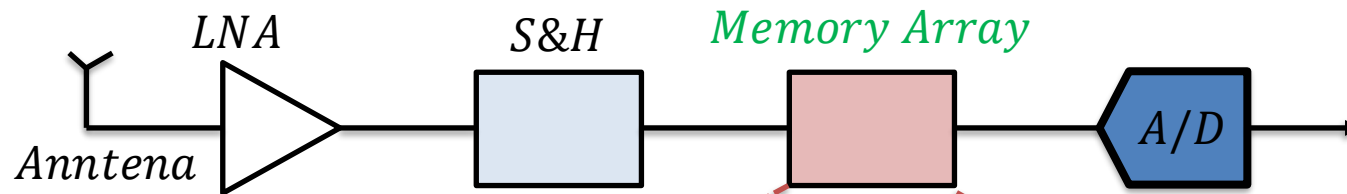


(b) Architecture of In-Memory Computing illustrating the integration of computation within memory arrays to minimize data movement and enhance processing efficiency.

ALU(Arithmetic Logic Unit)

**In memory & Von-Neumann
computing**

Problem Statement



Applications:

- RF receivers
- Radar systems
- High-speed communication
- Edge AI sensing
- Direct RF sampling

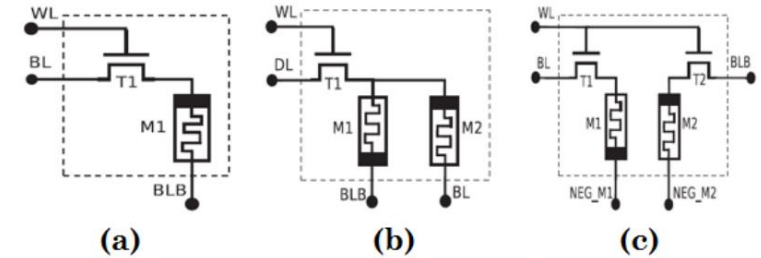
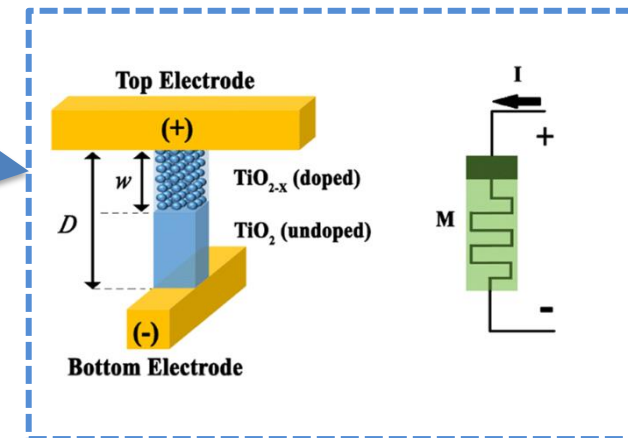
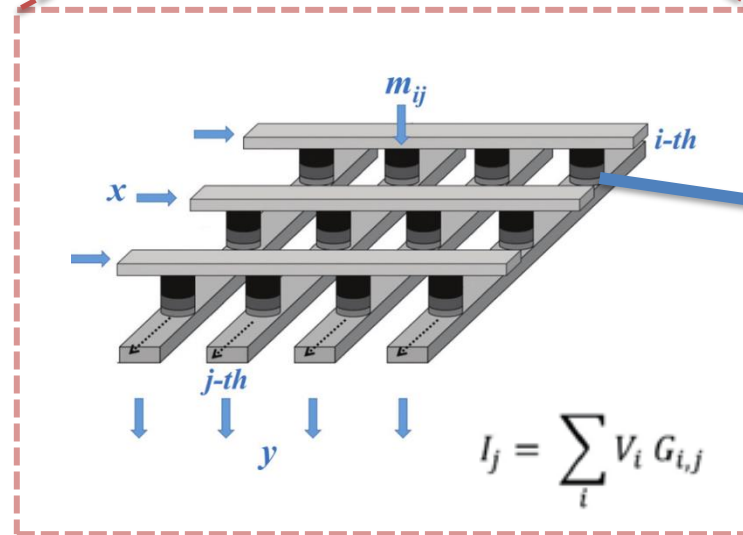
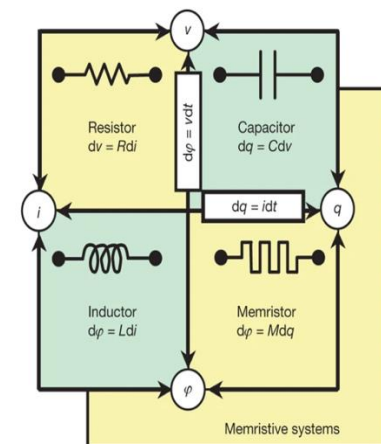


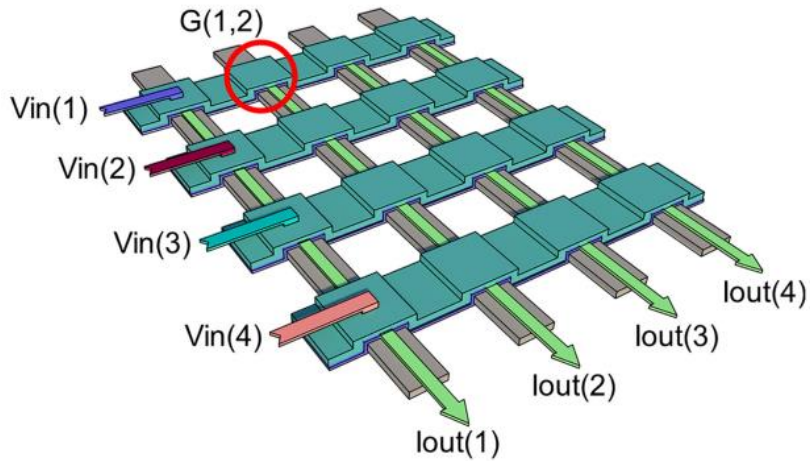
Fig. 16. Topologies of memristor-based RRAM cells. (a) 1T1M, (b) 1T2M, and (c) 2T2M [112].



Problem Statement

- **Wire resistance** → IR-drop
- **Capacitance/inductance** → distortions
- **Device variability** → drift, noise
- Example: outputs deviate significantly from ideal MVM

$$I_j = \sum_i V_i G_{i,j}$$



Crossbar Memristor

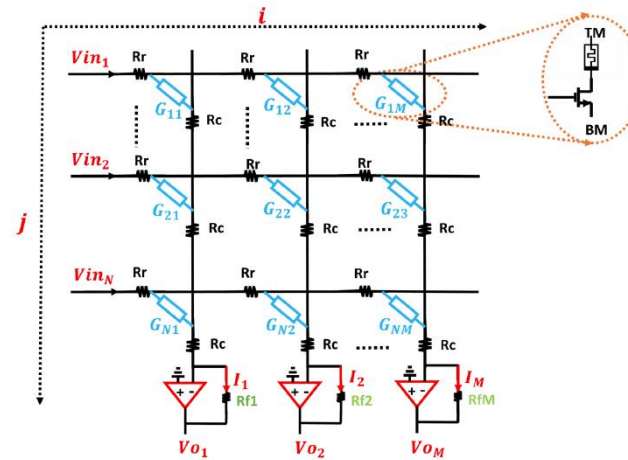
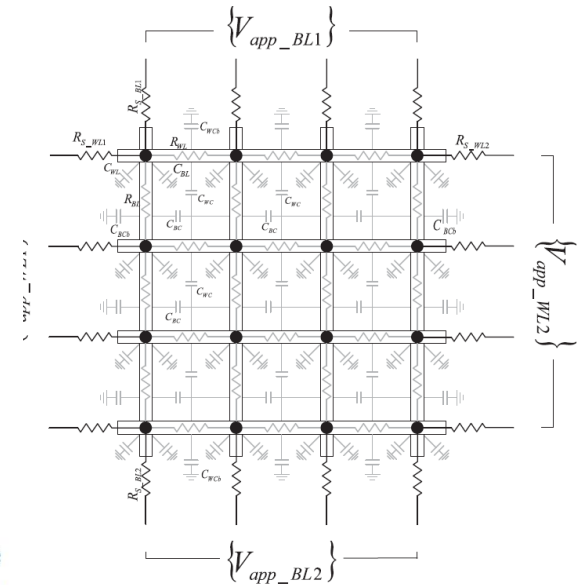


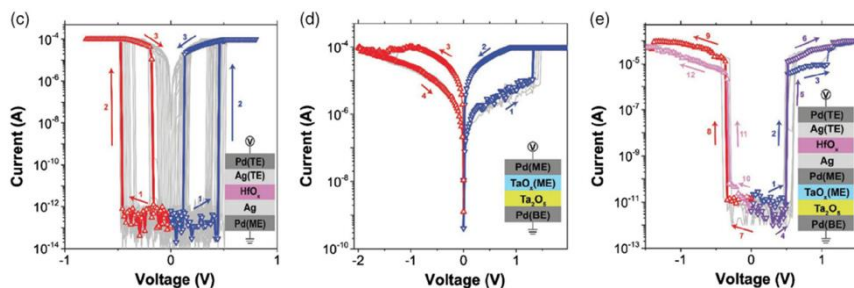
Figure 3: Memristor crossbar performing MVM. Conductance matrix G (size $N \times M$) is multiplied by voltage input vector V_{in} (size $1 \times N$).



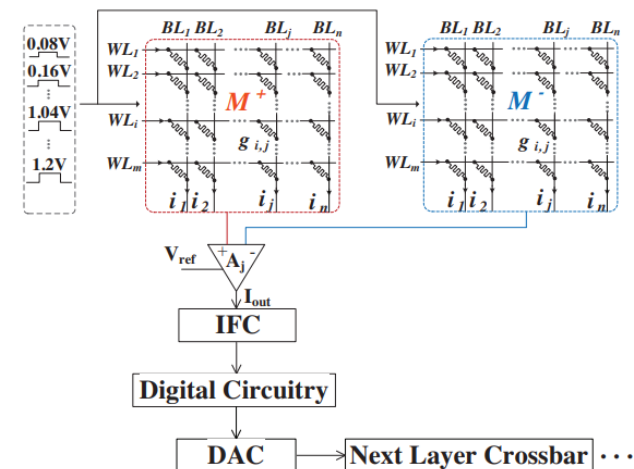
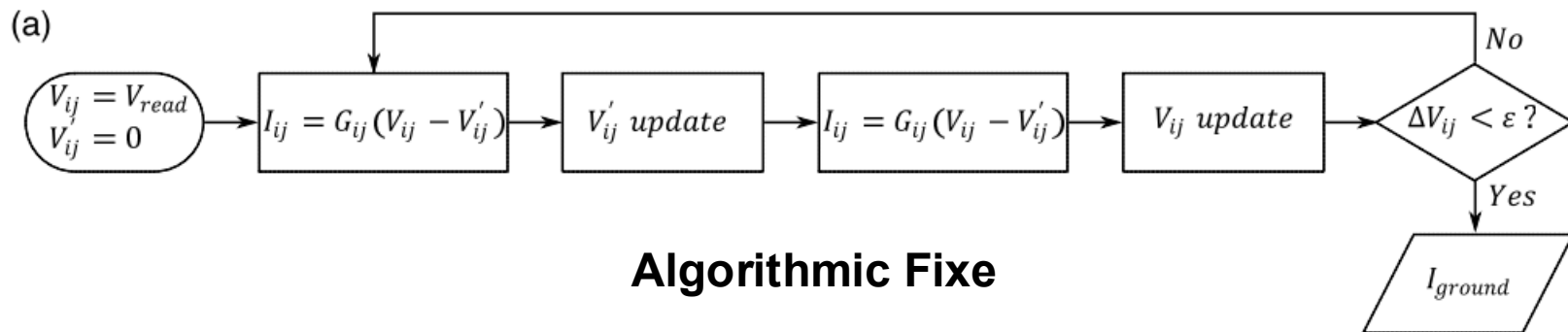
Model of a Crossbar Memristor

Related Work

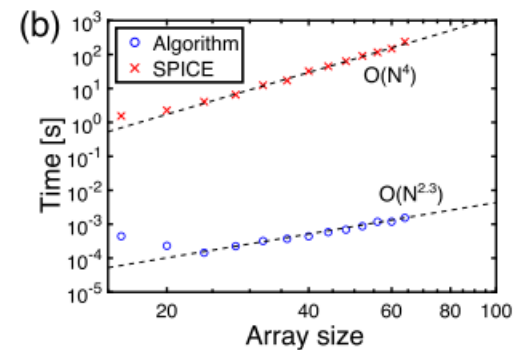
- **Hardware fixes** → bigger area, higher complexity
- **Circuit-level** → error correction, compensation
- **Algorithmic** → retraining with hardware-aware models
- Still: incomplete + trade-offs



Hardware Fixe



Circuit-level Fixe



Contributions

- **Post-fabrication calibration** (no hardware change)
 - **Neural network framework**
 - **accuracy improvement** (MSE reduction)
-

Our Approach

- Learn **systematic distortions** with NN
- Input:** Ideal conductance matrix (encode latent vector) , **Real measured currents**
- Output:** corrected, **near-ideal currents**

$$I_{\text{real}} = \mathcal{F}(M, V_{\text{in}}) + \varepsilon \quad (2)$$

$$\begin{aligned} \hat{I}_{\text{out}} &= \mathcal{G}(I_{\text{real}}, M) \equiv \mathcal{G}_{\theta}(I_{\text{real}}, M) = \tilde{I}_{\text{out}} \\ &\rightarrow \max_{\theta} p_{\theta}(\hat{I}_{\text{out}} | I_{\text{real}}, M) \\ &= \max_{\theta} p_{\theta}(\hat{I}_{\text{out}} | \mathcal{F}(M, V_{\text{in}}) + \varepsilon, M) \end{aligned} \quad (3)$$

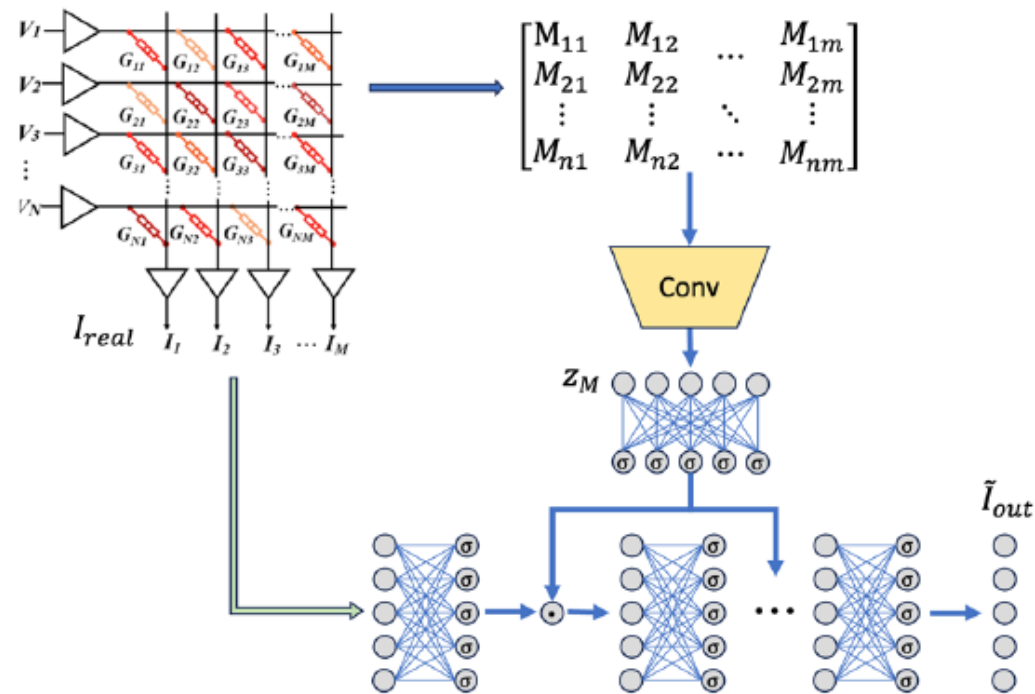


Figure 4: Model Structure

$$\begin{aligned} O_{i+1} &= CS(O_i, z_m) \\ &= (W_1 O_i + b_1) \odot \sigma(W_2 z_m + b_2) + W_3 z_m, \\ i &= 0, 1, 2, \dots, N-1, \end{aligned} \quad (4)$$

Framework

- Step 1: Encode conductance matrix $\rightarrow$ latent vector Z_m
- Step 2: Used with measured output I_{real}
- Step 3: Neural net calibration layers $\rightarrow I_{out}$
- Activation = sigmoid
- Non-linear inverse operator G_θ
- Compensate: IR-drop, noise, device non-linearity
- Lightweight inference (offline training, fast runtime)
- Adaptable to drift/ageing with small calibration set
- Training: random input vectors + random matrices
- Validation: sinusoidal input, FIR filter config

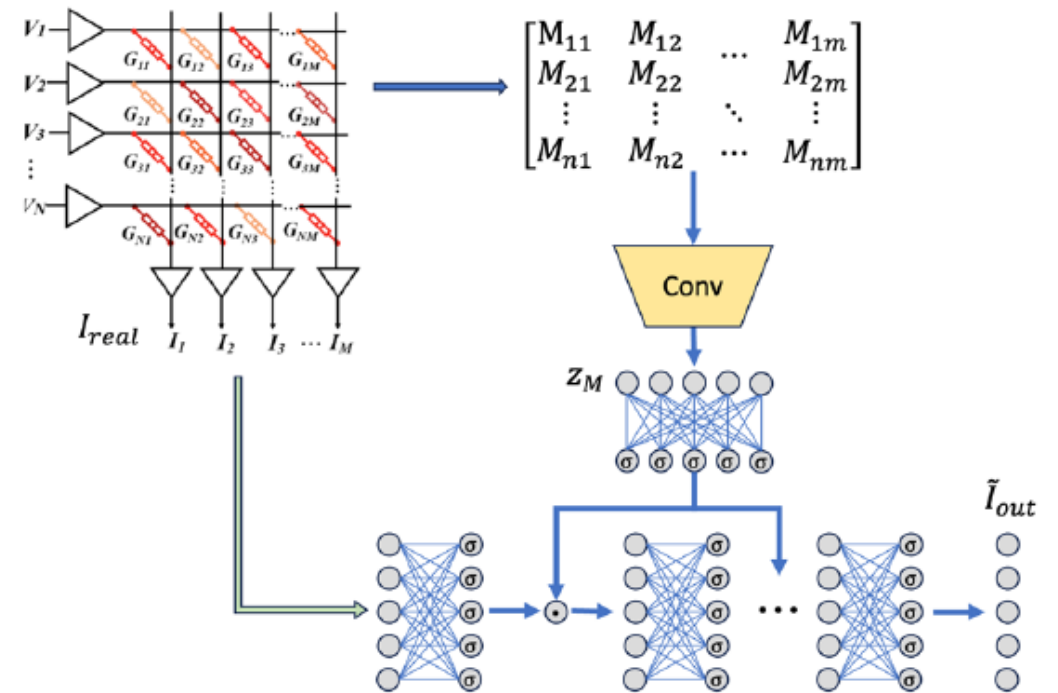
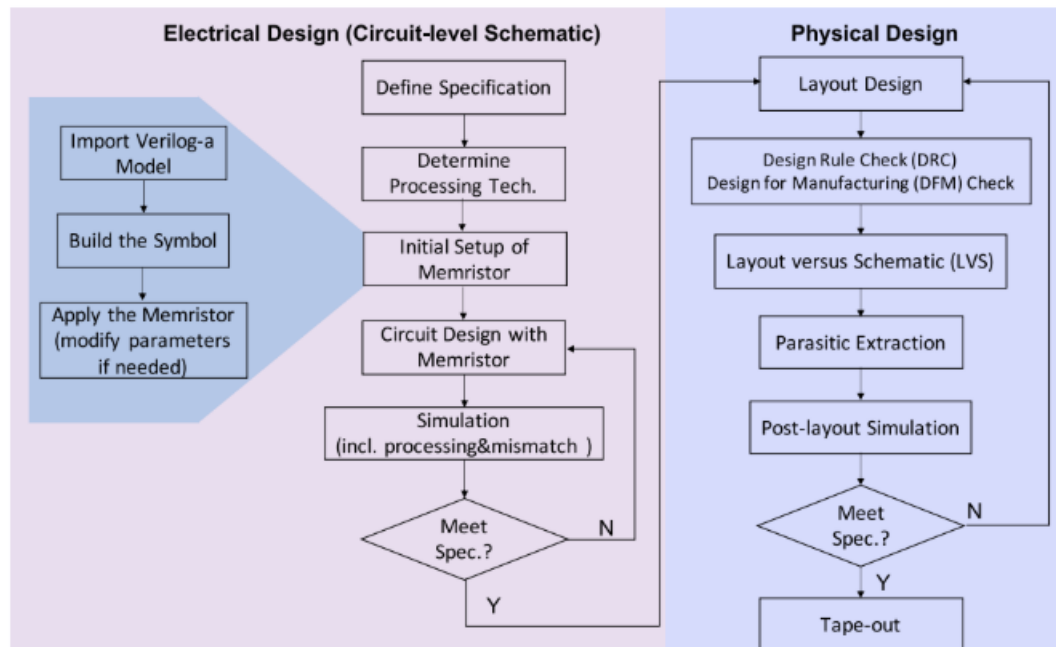


Figure 4: Model Structure

Data Collection

- Custom Verilog-A memristor model in Cadence Virtuoso
- Integrated with **22nm FD-SOI CMOS**
- Dataset: 1,000 conductance matrices ($16.6\mu\text{S}$ – $100\mu\text{S}$), 100 input vectors each (0.1 – 0.5V), $\rightarrow$ **100,000 VMM outputs**



```

1. module analytical (p, n);
2.   inout p, n;
3.   electrical p, n;
4.   parameter real Ap = 0.12340, An = -0.33000;
5.   parameter real tp = 2.74111, tn = 2.59685;
6.   parameter real rp0 = -40928.13784, rp1 = 55117.97865;
7.   parameter real m0 = 41366.35820, m1 = 7789.66771;
8.   parameter real Rinit = 40000;
9.   parameter real eta = 1;
10.  parameter real ap=0.225, bp=4.12;
11.  parameter real an=0.2801, bn=4.10;
12.  real Rmp, Rmn, svp, svn, vin, RS, IVp, IVn, IV;
13.  real first_iteration, R0_last, dt, it;
14.  analog function integer stp;
15.    real arg; input arg;
16.    stp = (arg >= 0 ? 1 : 0);
17.  endfunction
18.  analog begin
19.    if (first_iteration==0) begin
20.      it=0;      R0_last=Rinit;
21.    end
22.    dt=$abstime-it;
23.    vin=V(p,n);
24.    Rmp=rp0+rp1*vin;   Rmn=m0+m1*vin;
25.    if (vin>0) begin
26.      svp=Ap*(-1+exp(abs(vin)/tp));
27.      RS=(R0_last+svp*Rmp*(Rmp-R0_last)*dt)/(1+svp*(Rmp-R0_last)*dt);
28.    end
29.    else begin
30.      svn=An*(-1+exp(abs(vin)/tn));
31.      RS=(R0_last+svn*Rmn*(Rmn-R0_last)*dt)/(1+svn*(Rmn-R0_last)*dt);
32.    end
33.    if (RS>=Rmp && vin>0)      RS=R0_last;
34.    if (RS<=Rmn && vin<0)      RS=R0_last;
35.    if (abs(vin)<=0.5)          RS=R0_last;
36.    IVp=ap*(1/RS)*sinh(bp*vin);
37.    IVn=an*(1/RS)*sinh(bn*vin);
38.    IV=IVp*stp(vin)+IVn*stp(-vin);
39.    I(p, n)<+ IV;
40.    R0_last=RS;
41.    first_iteration=1;
42.    it=$abstime;
43.  end
44. endmodule
  
```

Results (Training/Validation)

- MSE reduction $\sim 1000\times$: Before calibration: 0.13 , After calibration: $1.4e-4$
- Strong generalization to unseen signals
- Works with sinusoidal + DC inputs

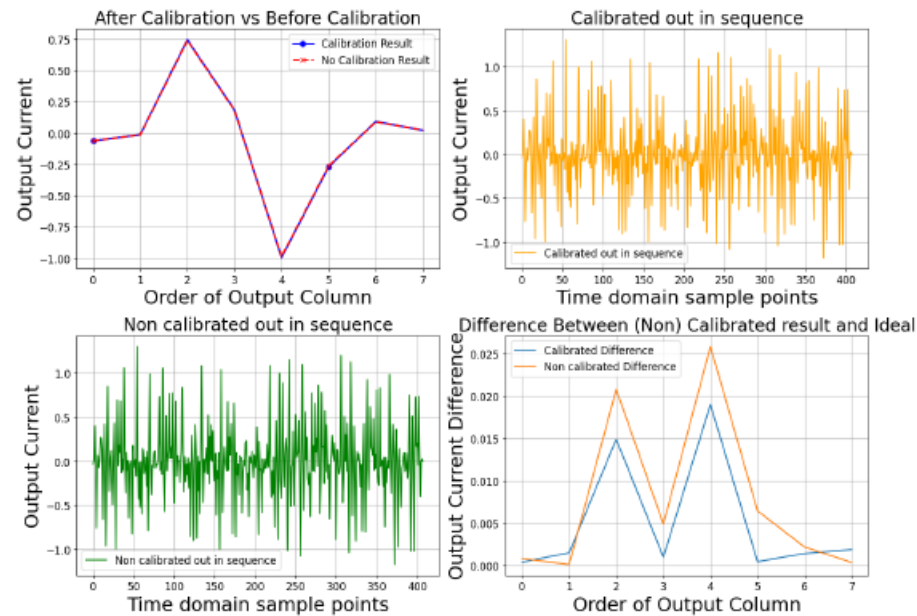


Figure 6: Visualization of the calibration results compared with crossbar original output.

Table 1: Training and Testing MSE Performance for Different Input Type

Input Data Type	$(\tilde{I}_{out} - I_{ideal})^2$	$(I_{real} - I_{ideal})^2$
Training with Random	0.000139	0.131878
Validating with Random	0.000149	0.122114
Testing with Sinusoidal	0.003398	0.032687
Testing with DC Input	0.003185	0.031794

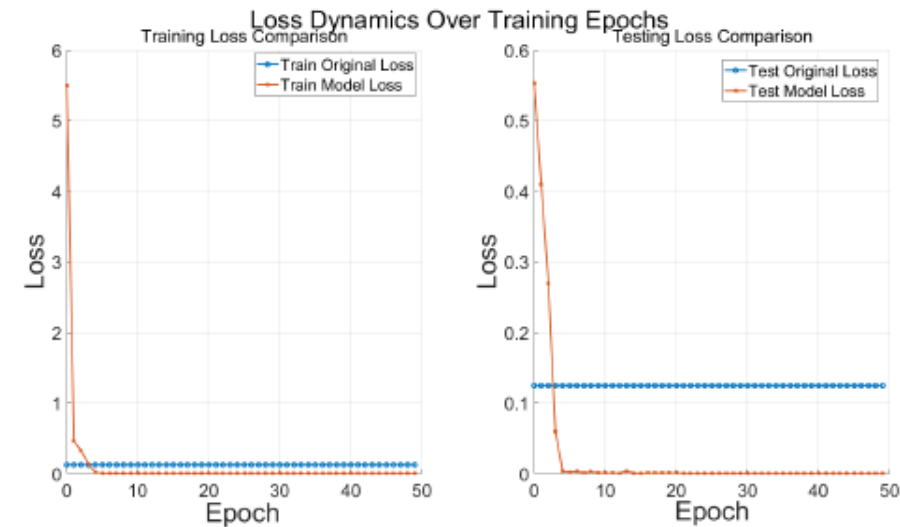


Figure 5: Training and Validating results

Robustness

- Handles parasitic distortions + noise
 - No frequent retraining needed
 - Low overhead → only inference at runtime
-

Comparison

- Device-level: area + complexity cost
 - Circuit-level: partial corrections only
 - Our method: post-fab, lightweight, scalable
-

Future Work

- Fabrication + Silicon validation Under noise, drift, temp.
 - Extend to **larger/tiled arrays** (hierarchical calibration)
 - Conditional fine-tuning for ageing effects
 - Public release: Verilog-A
-

Conclusion

- Neural network framework → restores memristor accuracy
 - Corrects distortions without hardware changes
 - Enables practical, high-precision IMC systems
 - Key step toward reliable neuromorphic computing
-

Acknowledgments / Q&A

- Intelligent Electromagnetic Sensor Lab (IEMSL), Texas A&M
 - Collaborators & advisors
 - Thank you
-